

ARQUITETURA ORIENTADA A EVENTOS EM NUVEM: UMA PROPOSTA BASEADA EM SERVIÇOS AWS PARA APLICAÇÕES ESCALÁVEIS

EVENT-DRIVEN ARCHITECTURE IN THE CLOUD: AN AWS-BASED PROPOSAL FOR SCALABLE APPLICATIONS

Pedro Vitor Ribeiro Silva^{1, i}
Deivison Shindi Takatu^{2, ii}

RESUMO

Aplicações *web* modernas precisam absorver picos de carga, evoluir continuamente e manter alta disponibilidade, requisitos que arquiteturas monolíticas atendem com dificuldade. Este trabalho, desenvolvido como projeto de Iniciação Científica, propõe uma Arquitetura Orientada a Eventos (*Event-Driven Architecture* – EDA) construída sobre serviços gerenciados da Amazon Web Services (AWS). A pesquisa é aplicada, qualitativa e exploratória: a partir de um levantamento bibliográfico sobre microsserviços, computação em nuvem e padrões de escalabilidade, definiram-se requisitos arquiteturais e projetou-se uma solução em cinco camadas – apresentação, entrada, mensageria, processamento e persistência/observabilidade – que integra CloudFront, API Gateway, Cognito, SNS, SQS, Lambda, DynamoDB, RDS, S3 e CloudWatch. A análise qualitativa indica que a mensageria assíncrona no modelo *publish/subscribe*, combinada a funções *serverless*, reduz o acoplamento, isola falhas e permite escalabilidade horizontal com custo proporcional ao uso. Como limitação, a proposta ainda não foi validada experimentalmente; a implementação de um protótipo e a medição de latência e custo constituem os próximos passos.

Palavras-chave: arquitetura orientada a eventos; computação em nuvem; *serverless*; microsserviços; escalabilidade.

ABSTRACT

Modern web applications must absorb load peaks, evolve continuously and remain highly available, requirements that monolithic architectures struggle to meet. This work, developed as an undergraduate research project, proposes an Event-Driven Architecture (EDA) built on managed services from Amazon Web Services (AWS). The research is applied, qualitative and exploratory: based on a literature survey on microservices, cloud computing and scalability patterns, architectural requirements were defined and a five-layer solution was designed – presentation, entry, messaging, processing and persistence/observability – integrating CloudFront, API Gateway, Cognito, SNS, SQS, Lambda, DynamoDB, RDS, S3 and CloudWatch. The qualitative analysis indicates that asynchronous publish/subscribe messaging combined with serverless functions reduces coupling, isolates failures and enables horizontal scalability with pay-per-use costs. As a limitation, the proposal has not yet been experimentally validated; implementing a prototype and measuring latency and cost are the next steps.

Keywords: event-driven architecture; cloud computing; serverless; microservices;

¹ Graduando em Tecnologia em Análise e Desenvolvimento de Sistemas no SENAI “Gaspar Ricardo Júnior”, Sorocaba-SP. E-mail: deivisontakatu@gmail.com

² Docente do SENAI “Gaspar Ricardo Júnior”. Mestre em Ciência da Computação (UFSCar). E-mail: deivisontakatu@gmail.com

scalability.

1 INTRODUÇÃO

A transformação digital ampliou a dependência das organizações em relação a sistemas de *software* distribuídos, que precisam responder em tempo real a volumes crescentes de usuários e dados. A popularização da computação em nuvem, com elasticidade, alta disponibilidade e cobrança sob demanda, viabilizou novos estilos arquiteturais (Rapôso; Costa Junior; Ferreira, 2024), e a migração de aplicações monolíticas para microsserviços tornou-se recorrente na indústria (Deprá *et al.*, 2021). Nesse cenário, a Arquitetura Orientada a Eventos (*Event-Driven Architecture* – EDA), na qual os componentes se comunicam por eventos publicados em um intermediário de mensagens, destaca-se por favorecer o desacoplamento e a resiliência (Vilaça; Azevedo, 2018). Este trabalho apresenta uma proposta de EDA sobre serviços gerenciados da Amazon Web Services (AWS).

1.1 Problema de pesquisa

Sistemas monolíticos exigem a reimplantação completa da aplicação a cada alteração, concentram pontos únicos de falha e só escalam como um bloco, o que gera gargalos e custos desproporcionais sob picos de demanda (Garcia, 2023). A questão que orienta a pesquisa é: como estruturar, em nuvem pública, uma arquitetura de *software* escalável, resiliente e desacoplada, capaz de processar eventos de forma assíncrona sem impor à equipe de desenvolvimento um esforço operacional elevado?

1.2 Objetivo(s)

O objetivo geral é propor e descrever uma arquitetura orientada a eventos, baseada em serviços gerenciados da AWS, que atenda a requisitos de escalabilidade, desempenho e resiliência. Os objetivos específicos são: (i) identificar, na literatura, padrões arquiteturais aplicáveis a sistemas escaláveis; (ii) descrever os componentes da solução e o papel de cada um; (iii) discutir os mecanismos de segurança e escalabilidade adotados; e (iv) consolidar documentação técnica reaproveitável. Parte-se da hipótese de que mensageria assíncrona e funções *serverless* proporcionam ganhos de escalabilidade em relação a abordagens monolíticas e síncronas, sem complexidade operacional incompatível com equipes de médio porte.

1.3 Justificativa

No levantamento realizado, a literatura nacional concentra-se na decomposição de monolitos e na adoção de microsserviços; são poucos os trabalhos que integram, em uma única proposta, EDA, serviços *serverless* gerenciados e a discussão conjunta de segurança, escalabilidade e observabilidade. Uma referência arquitetural aberta e documentada auxilia equipes pequenas e médias a tomar decisões fundamentadas e serve de material de estudo para estudantes de graduação.

2 REVISÃO DE LITERATURA

A computação em nuvem é um modelo de provisionamento de recursos computacionais sob demanda, acessíveis pela rede, com elasticidade e cobrança por uso. Rapôso, Costa Junior e Ferreira (2024) apontam que sua adoção promove mudanças estruturais nas arquiteturas de *software*, tornando escalabilidade horizontal, modularidade e resiliência atributos centrais dos sistemas.

A arquitetura de microsserviços decompõe a aplicação em serviços pequenos e autônomos, com implantação independente. Em levantamento com 23 empresas de *software* de Maringá-PR, Deprá *et al.* (2021) verificaram que escalabilidade, manutenção e evolução foram os principais motivos de migração para esse estilo, citados por 77,8% das empresas que o adotam. A revisão multivocal de Moraes, Cordeiro e Oliveira Jr (2025) mapeia os tópicos, desafios e tecnologias associados ao tema. Os ganhos, porém, têm contrapartida: ao comparar os modelos monolítico e de microsserviços no setor financeiro, Costa e Antonelli (2025) observam que modularidade e manutenção são beneficiadas, mas o desacoplamento exige maior atenção à infraestrutura e à latência.

Na EDA, a comunicação ocorre pela produção e pelo consumo de eventos, com três papéis: o produtor, o intermediário (*broker*) e o consumidor. O desacoplamento temporal e espacial promovido pelo *broker* permite que os serviços evoluam de forma independente, processem mensagens em paralelo e reprocessem eventos em caso de erro. Villaça e Azevedo (2018) aplicam esse estilo a microsserviços para integrar dados frequentemente atualizados oriundos de bases políglotas.

Quanto à escalabilidade, Porto (2009) sistematiza padrões e diretrizes arquiteturas para o escalonamento horizontal, e Garcia (2023) destaca o uso de filas, particionamento, replicação e *cache* em sistemas de grande escala. Esses princípios são compatíveis com funções *serverless*, que não mantêm estado entre invocações.

3 METODOLOGIA

Trata-se de pesquisa de natureza aplicada, abordagem qualitativa e caráter exploratório, conduzida em quatro etapas. (1) Levantamento bibliográfico: foram reunidos 16 trabalhos em língua portuguesa (artigos de periódicos e de eventos da SBC, dissertações, trabalhos de conclusão de curso e literatura técnica) sobre arquitetura de *software*, microsserviços, computação em nuvem, escalabilidade e qualidade de *software*. (2) Definição de requisitos: a partir do problema, estabeleceu-se que a solução deve suportar picos de carga sem indisponibilidade, permitir a evolução independente dos componentes, reduzir o acoplamento entre módulos e aproveitar serviços gerenciados para diminuir o esforço operacional. (3) Projeto arquitetural: selecionaram-se os serviços AWS que atendem a cada requisito e a solução foi documentada em descrição textual e diagrama. (4) Análise qualitativa: as decisões de projeto foram confrontadas com os padrões de segurança e escalabilidade descritos na literatura.

Todos os artefatos estão versionados em repositório público no GitHub (<https://github.com/Pedro-Vitor-Ribeiro-Silva/IniciacaoCientifica>). Nesta fase não foram realizadas implementação nem medições de desempenho.

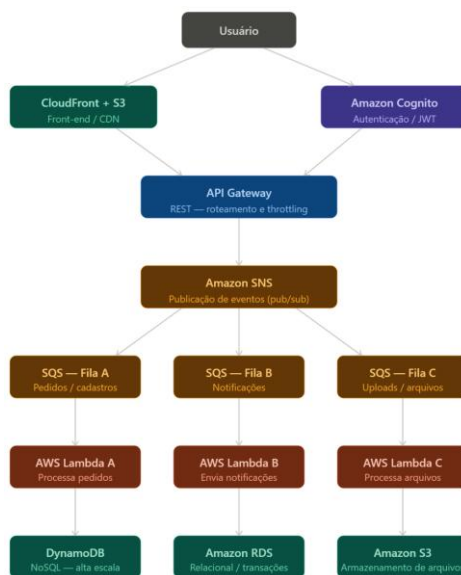
4 RESULTADOS E DISCUSSÕES

O principal resultado é a arquitetura apresentada na Figura 1, implantada integralmente sobre serviços gerenciados e organizada em cinco camadas: apresentação, entrada, mensageria, processamento e persistência/observabilidade.

Na apresentação, a interface em React/Next.js é hospedada no Amazon S3 e distribuída pelo CloudFront, com o AWS WAF na borda. Na entrada, o API Gateway realiza roteamento e *throttling* e valida os *tokens* JWT emitidos pelo Amazon Cognito. Requisições simples são atendidas de forma síncrona por funções AWS

Lambda; operações de maior latência tornam-se eventos de domínio (por exemplo, “pedido criado” ou “arquivo enviado”) publicados em tópicos do Amazon SNS. Filas do Amazon SQS assinam esses tópicos e entregam as mensagens a funções Lambda especializadas, cada uma responsável por um único tipo de evento, que persistem dados no DynamoDB (operacionais, de alta vazão) ou no RDS (transacionais), gravam arquivos no S3 por meio de URLs pré-assinadas e enviam notificações pelo Amazon SES. CloudWatch, X-Ray e CloudTrail concentram métricas, *logs*, rastreamento e auditoria.

Figura 1 – Arquitetura orientada a eventos proposta



Fonte: elaborado pelo autor (2026).

Quanto à escalabilidade, o Lambda escala por execuções concorrentes, o DynamoDB ajusta a capacidade no modo sob demanda e as filas SQS funcionam como amortecedor, evitando que produtores rápidos sobrecarreguem consumidores lentos – aplicação direta dos padrões de filas e de componentes sem estado descritos por Porto (2009) e Garcia (2023). Como a cobrança ocorre por uso, o custo acompanha a demanda. Quanto à resiliência, a falha de um consumidor não se propaga aos demais: as mensagens permanecem na fila, são reprocessadas e, após tentativas sem sucesso, seguem para *Dead Letter Queues* para análise posterior. A segurança é construída em camadas: WAF no perímetro, autenticação federada no Cognito, políticas de menor privilégio no AWS IAM e criptografia em trânsito (TLS) e em repouso (AWS KMS).

A discussão também evidencia contrapartidas. A consistência passa a ser eventual e a entrega “ao menos uma vez” exige consumidores idempotentes; o rastreamento de um fluxo distribuído depende da camada de observabilidade, em linha com o alerta de Costa e Antonelli (2025) sobre infraestrutura e latência; e a opção por um único provedor implica dependência tecnológica (*vendor lock-in*). Por fim, os benefícios apontados decorrem de análise qualitativa, e não de medições: a hipótese permanece sustentada teoricamente, mas ainda não verificada de forma empírica.

5 CONCLUSÃO

O objetivo geral foi alcançado: propôs-se e documentou-se uma arquitetura orientada a eventos sobre serviços gerenciados da AWS, com a descrição dos componentes,

do fluxo de dados e dos mecanismos de segurança e escalabilidade, cumprindo os quatro objetivos específicos. A hipótese de que mensageria assíncrona e funções *serverless* trazem ganhos de escalabilidade sem complexidade operacional excessiva mostrou-se coerente com a literatura consultada, porém foi confirmada apenas no plano conceitual.

As principais limitações são a ausência de validação experimental e o foco em um único provedor de nuvem. Como trabalhos futuros, pretende-se: (i) implementar um protótipo funcional; (ii) comparar custo e latência com uma arquitetura monolítica equivalente, sob carga controlada; (iii) avaliar a portabilidade para outros provedores; e (iv) incorporar práticas de FinOps para otimização contínua de custos.

REFERÊNCIAS

- COSTA, R. F.; ANTONELLI, H. L. Modernização de arquitetura de sistemas: uma abordagem para transformação digital no sistema financeiro. *In: ESCOLA REGIONAL DE INFORMÁTICA DO ESPÍRITO SANTO (ERI-ES)*, 10., 2025. **Anais [...]**. Porto Alegre: SBC, 2025. p. 140-149. DOI: 10.5753/eries.2025.15930. Disponível em: <https://sol.sbc.org.br/index.php/eries/article/view/38408>. Acesso em: 10 jun. 2026.
- DEPRÁ, T. G. B. *et al.* Estudo exploratório sobre o uso da arquitetura de microsserviços na indústria de software. **Revista Eletrônica de Iniciação Científica em Computação**, Porto Alegre, v. 19, n. 3, 2021. DOI: 10.5753/reic.2021.1999. Disponível em: <https://journals-sol.sbc.org.br/index.php/reic/article/view/1999>. Acesso em: 10 jun. 2026.
- GARCIA, V. C. **Padrões de arquitetura para escalabilidade**: uma perspectiva prática. Recife, 2023. Disponível em: <https://viniciusgarcia.me/architecture/padrees-de-rqitetura-para-escalabilidade/>. Acesso em: 10 jun. 2026.
- MORAIS, L. Z.; CORDEIRO, A. F. R.; OLIVEIRAJR, E. Arquitetura de microsserviços: uma revisão multivocal. *In: ESCOLA REGIONAL DE ENGENHARIA DE SOFTWARE (ERES)*, 2025. **Anais [...]**. Porto Alegre: SBC, 2025. Disponível em: <https://sol.sbc.org.br/index.php/eres/article/view/40375>. Acesso em: 10 jun. 2026.
- PORTO, I. O. **Padrões e diretrizes arquiteturas para escalabilidade de sistemas**. 2009. Dissertação (Mestrado em Ciência da Computação) – Universidade Federal de Uberlândia, Uberlândia, 2009. Disponível em: <https://repositorio.ufu.br/bitstream/123456789/12488/1/ivens.pdf>. Acesso em: 10 jun. 2026.
- RAPÔSO, C. F. L.; COSTA JUNIOR, O. G.; FERREIRA, A. S. Impactos da computação em nuvem na arquitetura de software: uma análise de literatura. **Revista Tópicos**, [s. l.], 2024. DOI: 10.5281/zenodo.13949416. Disponível em: <https://revistatopicos.com.br/artigos/impactos-da-computacao-em-nuvem-na-arquitetura-de-software-uma-analise-de-literatura>. Acesso em: 10 jun. 2026.
- VILLAÇA, L.; AZEVEDO, L. Uma arquitetura de microsserviços orientada a eventos para integração de dados frequentemente atualizados oriundos de bases de dados poliglotas. *In: SIMPÓSIO BRASILEIRO DE SISTEMAS DE INFORMAÇÃO (SBSI)*, 14., 2018. **Anais Estendidos [...]**. Porto Alegre: SBC, 2018. p. 100-104. Disponível em: https://sol.sbc.org.br/index.php/sbsi_estendido/article/view/6205. Acesso em: 10

jun. 2026.

AGRADECIMENTOS

Ao SENAI “Gaspar Ricardo Júnior” e ao professor Deivison Shindi Takatu, pela orientação e pelo acompanhamento ao longo do projeto de Iniciação Científica.

SOBRE O(S)AUTOR(ES)

Sobre os autores:

i PEDRO VITOR RIBEIRO SILVA



Possui formação técnica pelo SENAI “Luiz Pagliato” (2024–2025), na qual recebeu o Prêmio Destaque da Turma, e cursa atualmente o Tecnólogo em Análise e Desenvolvimento de Sistemas no SENAI “Gaspar Ricardo Júnior”, em Sorocaba-SP (2026–2027). Tem experiência em desenvolvimento *full-stack*, com ênfase em Java/Spring Boot, Python, TypeScript/React e bancos de dados PostgreSQL. Atua na empresa NaNet Construção, do setor de comércio eletrônico, nas áreas de desenvolvimento de sistemas e de operações de *e-commerce*.

ii DEIVISON SHINDI TAKATU



Possui graduação em Análise e Desenvolvimento de Sistemas (2016) e em Gestão Empresarial (2018) pela Faculdade de Tecnologia de Itapetininga, Licenciatura em Matemática pela UNIVESP (2023) e graduação em Ciências Econômicas pela UNOPAR (2023). É especialista em Informática Aplicada à Educação pelo IFSP (2018) e Mestre em Ciência da Computação pela UFSCar, campus Sorocaba (2021). Atualmente é docente do SENAI “Gaspar Ricardo Júnior”, em Sorocaba-SP, onde orienta a disciplina de Iniciação Científica do curso de Tecnologia em Análise e Desenvolvimento de Sistemas. Tem experiência na área de Ciência da Computação, com ênfase em engenharia de *software*, informática na educação e robótica educacional.